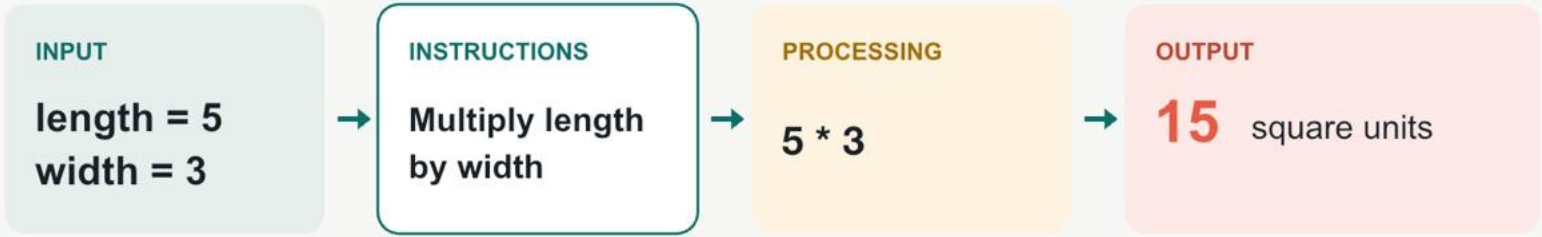
# A program transforms input into output

A program is a set of instructions that a computer can execute.




**The computer follows the instructions exactly. It does not infer missing steps.**

# An algorithm is an ordered plan

An algorithm is a finite, ordered sequence of steps for solving a problem.

1. **Read the length**
2. **Read the width**
3. **Multiply length by width**
4. **Display the area**

WHY ORDER MATTERS

**Calculate the area**

before reading length and width?

**The values do not exist yet.**

Plan in plain English first.

# Python runs statements from top to bottom

**Statement** = one code instruction **State** = values remembered now **Trace** = follow each line

PROGRAM

```
length = 5
width = 3
area = length * width
print(area)
```

**State after each line**

length: 5

length: 5 width: 3

length: 5 width: 3 area: 15

**output: 15**

**Stored values are program state. Printed text is output.**

# A type tells Python what a value means

**int**

**5**

**Integer**

A whole number

**float**

**3.5**

**Float**

A decimal number

**str**

**"Rectangle"**

**String**

Text in quotes

**bool**

**True**

**Boolean**

True or False

**5** is a number **"5"** is text **Type controls valid operations.**

```
type(5) type(3.5) type("Rectangle") type(True)
```

# Variables give values meaningful names

A variable is a name that refers to a value.

**length** = **5**

variable name — assign — value

### Reassignment changes the value

length = 5 → length = 7

### Do not confuse

= assigns a value

== compares values

**Clear: rectangle_length**

Vague: x

# Expressions calculate new values

An expression combines values, variables, and operators to produce a result.

### AREA EXPRESSION

**length * width**

5 * 3

**15**

### PARENTHESES MAKE ORDER CLEAR

**(length + 2) * width**

(5 + 2) * 3

7 * 3

**21**

### OPERATOR REFERENCE

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| + | add | - | subtract | * | multiply | / | divide | // | whole | % | remainder |

# Comparisons produce True or False

A comparison asks a question about two values and returns a boolean.

**Start with area = 15**


| | |
|---|---|
| area > 20 | False |
| area == 15 | True |
| area != 10 | True |

**== equal** **!= different** **> greater** **< smaller** **>= at least** **<= at most**

**Week 2 uses these results to decide which code should run.**

# Strings and functions create readable output

**String** = text inside quotation marks. **Function** = reusable code that performs a task.
We call a function by writing its name followed by parentheses: **print(...)**

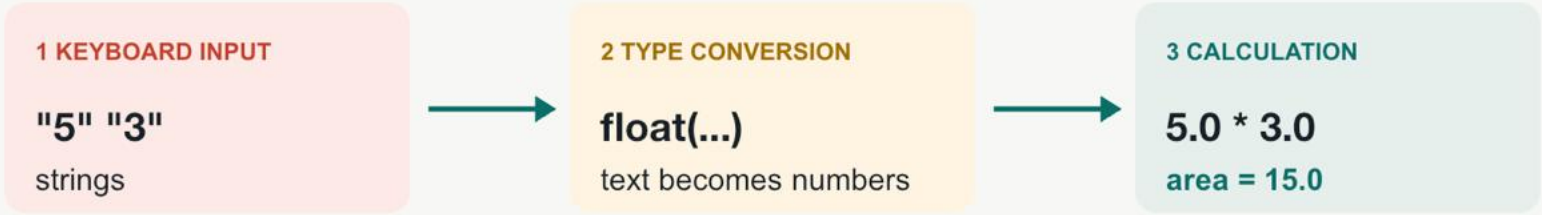
```
shape = "Rectangle" length = 5 width = 3
area = length * width
print(f"{shape}: {length} x {width} = {area} square units")
```

| {shape} | {length} | {width} | {area} |
|---|---|---|---|
| Rectangle | 5 | 3 | 15 |

**Rectangle: 5 x 3 = 15 square units**

# User input arrives as text

**input()** always returns a string, even when the user types digits.




```
length_text = input("Length: ") width_text = input("Width: ")
length = float(length_text) width = float(width_text)
area = length * width print(f"Area: {area} square units")
```

# Errors fail in different ways

### Syntax error

Python cannot start.

```
shape = "Rectangle
```

Missing closing quote

CHECK: PYTHON GRAMMAR

### Runtime error

The program starts, then fails.

```
float("wide")
```

Text is not a number

CHECK: INPUT AND DATA

### Logic error

The program runs, but the result is wrong.

```
area = length + width
```

Wrong operation

CHECK: REASONING

**Correct:** area = length * width  Output: 15  Read the final error line first.

# Convert minutes into hours and minutes

Use **//** for complete groups and **%** for the remainder.

### 135 minutes


135 // 60 = 2

135 % 60 = 15

**2 h 15 min**

```
minutes_text = input("Total minutes: ")
total_minutes = int(minutes_text)

hours = total_minutes // 60
minutes = total_minutes % 60

print(f"{hours} h {minutes} min")
```

### TEST BOUNDARIES, NOT JUST ONE EXAMPLE

| | | | | |
|---|---|---|---|---|
| 0 -> 0 h 0 min | 59 -> 0 h 59 min | 60 -> 1 h 0 min | 61 -> 1 h 1 min | 135 -> 2 h 15 min |

# Trace the program before you run it

**INPUT: total_minutes = 150**

```
hours = total_minutes // 60
minutes = total_minutes % 60
is_long = total_minutes >= 120
print(hours, minutes, is_long)
```

**2 30 True**

## You are ready when you can:

- recognize four basic types
- explain variables and expressions
- trace code in execution order
- separate input, state, and output
- identify three error categories

**Next: conditions, loops, functions, lists, and dictionaries** →